

UNIT - IV

4

Pushdown Automata (PDA)

Syllabus

Introduction, Formal definition of PDA, Equivalence of Acceptance by Final State and Empty stack, Non-deterministic PDA (NPDA), PDA and Context Free Language, Equivalence of PDA and CFG, PDA vs CFLs. Deterministic CFLs.

Contents

- 4.1 Formal Definition of PDA **Dec.-17,** Marks 4
- 4.2 Instantaneous Description **May-07,11,14,15,16,18,19,**
..... **Dec.-13,14,16,18,19,** Marks 16
- 4.3 Acceptance by Final State
- 4.4 Acceptance by Empty Stack. **Dec.-18, 19,** Marks 4
- 4.5 Equivalence of Acceptance by Final State and Empty Stack
..... **Dec.-09,13,** Marks 18
- 4.6 Non-deterministic PDA (NPDA) **Dec.-17, May-18,** Marks 6
- 4.7 PDA and Context Free Language **Dec.-05,08,11,14,16,17,18,19,**
..... **May-14,15,16,18,** Marks 12
- 4.8 Deterministic CFLs
- 4.9 Fill in the Blanks
- 4.10 Multiple Choice Questions

4.1 Formal Definition of PDA

SPPU : Dec.-17, Marks 4

The push down automata will have input tape, finite control and stack.

The input tape is divided in many cells. At each cell only one input symbol is placed thus certain input string is placed on tape. The finite control has some pointer which point the current symbol which is to be read. At the end of input \$ or Δ (blank) Δ (blank) symbol is placed which indicates end of input.

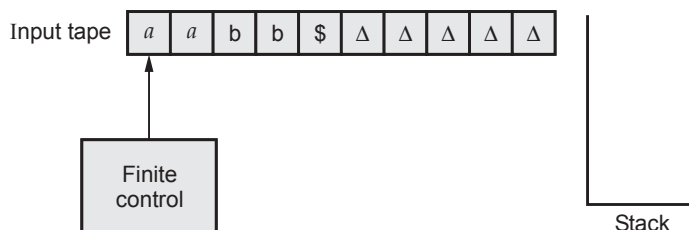


Fig. 4.1.1 Push down automata

The stack is such a structure in which you can push and remove the items from one end only. For example, if you place some coins one above the other, then a stack of coins is formed. While removing a coin we can only remove the coin which is at the top. Similarly while adding more coins to the stack we can place a new coin only on the topmost coin. Thus the stack of coin shows clear cut use of only one end of the stack.

In the push down automata, we are using the stack for storing the items temporarily. Inserting the symbol onto the stack is called push operation and removing a symbol from stack is called pop operation.

Let us have a formal definition of Push Down Automata (PDA).

The PDA can be defined as a collection of seven components.

1. The finite set of states Q .
2. The input set Σ .
3. Γ is a stack alphabet.
4. q_0 is initial stage, $q_0 \in Q$.
5. Z_0 is a start symbol which is in Γ .
6. Set of final states $F \subseteq Q$.
7. δ is mapping function used for moving from current state to next state.

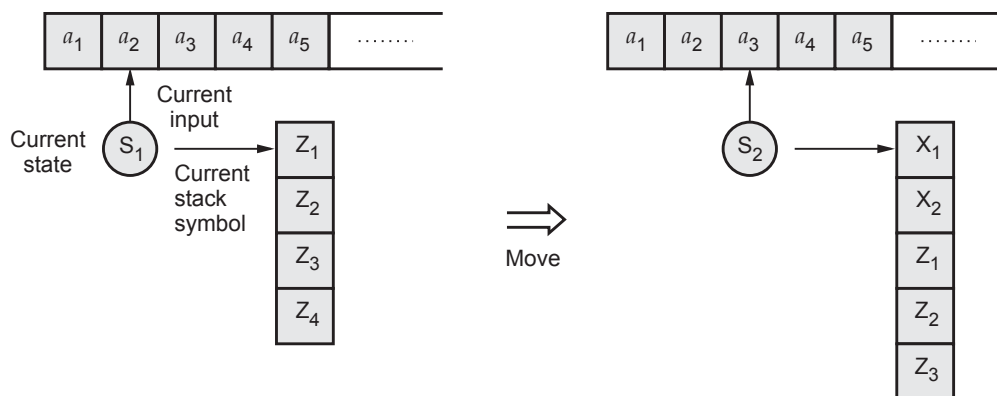
Example 4.1.1 Differentiate between FA and PDA.**SPPU : Dec.-17, End Sem. Marks 4****Solution :**

Sr. No.	FA	PDA
1.	The Finite Automata (FA) has no memory. Hence it can not remember already read input.	The push-down machine has a capacity of remembering already read input by means of stack.
2.	It accepts regular language	It accepts regular non-regular and context free languages
3.	It makes use of states and transitions to model any language	It makes use of push and pop operations to model any language
4.	It is simple to represent	It is more powerful than FSM

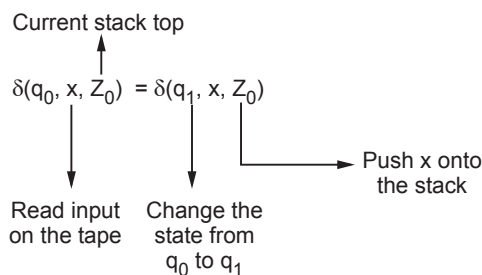
4.2 Instantaneous Description

SPPU : May-07,11,14,15,16,18,19, Dec.-13,14,16,18,19, Marks 16

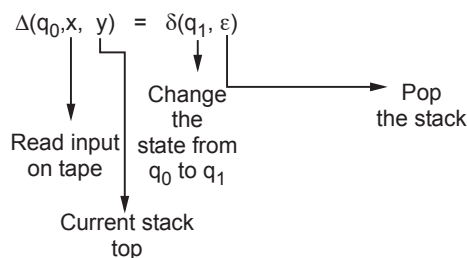
We can describe this behaviour of PDA by means of instantaneous description. It is given by ID. The moves of ID's are as shown below,

**Fig. 4.2.1**

From Fig. 4.2.1, if we are reading the current symbol a_2 , at current state S_1 and current stack symbol Z_1 then after a move we will reach to state S_2 and there will be some new symbol on the top of the stack. This description can be represented as -



(a) Push operation



(b) Pop operation

Fig. 4.2.2 Instantaneous description

Now let us solve some examples based on it.

Example 4.2.1 Design a PDA for accepting a language $\{L = a^n b^n \mid n \geq 1\}$.

SPPU : May-16, End Sem. Marks 7, Dec.-16, End Sem. Marks 6, May-18,19, End Sem, Marks 6

Solution : This is a language in which equal number of a 's are followed by equal number of b 's. The logic for this PDA can be applied as : First we will push all a 's onto the stack. Then on reading every single b each a is popped from the stack. If we read all b and remove all a 's and if we get stack empty then that string will be accepted. The instantaneous description can be given as -

$$\delta(q_0, a, Z_0) = (q_0, aZ_0)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, Z_0) = (q_2, \epsilon)$$

where q_0 is a start state and q_2 is accept state.

We will simulate this PDA for following string -

$$(q_0, aaabbb, Z_0) \vdash (q_0, aabbb, aZ_0)$$

$$\vdash (q_0, abbb, aaZ_0)$$

$$\vdash (q_0, bbb, aaaZ_0)$$

$$\vdash (q_1, bb, aaZ_0)$$

$$\vdash (q_1, b, aZ_0)$$

$$\vdash (q_1, \epsilon, Z_0)$$

$$\vdash (q_2, \epsilon)$$

ACCEPT state.

Example 4.2.2 Construct PDA for the language $L = \{a^n b^{2n} \mid n \geq 1\}$.

SPPU : May-11, Marks 7, Dec.-18, End Sem, Marks 6

Solution : In this language n number of a 's should be followed by $2n$ number of b 's. Hence we will apply a very simple logic and that is if we read single ' a ' we will push two a 's onto the stack. As soon as we read ' b ' then for every single ' b ' only one ' a ' should get popped from the stack. This basically maintains the $a^n b^{2n}$ count and sequence. The ID can be constructed as follows -

$$\delta(q_0, a, Z_0) = (q_0, aaZ_0)$$

$$\delta(q_0, a, a) = (q_0, aaa)$$

Now when we read b we will change the state from q_0 to q_1 and start popping corresponding ' a '. Hence,

$$\delta(q_0, b, a) = (q_1, \epsilon)$$

Thus this process of popping will be repeated unless all the symbols are read. Note that popping action occurs in state q_1 only.

$$\therefore \delta(q_1, b, a) = (q_1, \epsilon)$$

After reading all b 's all the corresponding a 's should get popped. Hence when we read ϵ as input symbol there should be nothing in the stack. Hence the move will be -

$$\delta(q_1, \epsilon, Z_0) = (q_2, \epsilon)$$

$$\text{where the PDA } P = (\{q_0, q_1, q_2\}, \{a, b\}, \{a, Z_0\}, \delta, q_0, Z_0, \{q_2\})$$

We can summarize the ID as

$$\delta(q_0, a, Z_0) = (q_0, aaZ_0)$$

$$\delta(q_0, a, a) = (q_0, aaa)$$

$$\delta(q_0, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, Z_0) = (q_2, \epsilon)$$

Let us simulate this PDA for some input string "aaabbbbbbb".

$$\begin{aligned} \delta(q_0, aaabbbbbbb, Z_0) &\vdash (q_0, aabbbbbbb, aaZ_0) \\ &\vdash (q_0, abbbbbbb, aaaaZ_0) \\ &\vdash (q_0, bbbbbbb, aaaaaaZ_0) \\ &\vdash (q_1, bbbbbbb, aaaaaaZ_0) \\ &\vdash (q_1, bbbbb, aaaaaZ_0) \\ &\vdash (q_1, bbbb, aaaaZ_0) \\ &\vdash (q_1, bbb, aaaZ_0) \\ &\vdash (q_1, bb, aaZ_0) \\ &\vdash (q_1, b, aZ_0) \\ &\vdash (q_1, \epsilon, Z_0) \\ &\vdash (q_2, \epsilon) \end{aligned}$$

Final state or ACCEPT state.

Thus input gets accepted by using the constructed PDA.

Example 4.2.3 Construct the PDA and then its equivalent CFG for the language L as given below :

$$L = \{XcX^r \mid X \in \{a, b\}^* \text{ and string } X^r \text{ is the reverse of string } X\}$$

SPPU : May-14, Marks 16, Dec.-19, End Sem, Marks 6

Solution : For a given PDA,

$$M = (\{q_0, q_1, q_2\}, \{a, b, c\}, \{a, b, Z_0\}$$

$\delta, q_0, Z_0, \{q_2\})$, the mapping function δ is given as

$$\begin{aligned} R_1 : \quad \delta(q_0, a, Z_0) &= \delta(q_0, aZ_0) \\ R_2 : \quad \delta(q_0, b, Z_0) &= \delta(q_0, bZ_0) \\ R_3 : \quad \delta(q_0, a, a) &= \delta(q_0, aa) \\ R_4 : \quad \delta(q_0, b, a) &= \delta(q_0, ba) \\ R_5 : \quad \delta(q_0, a, b) &= \delta(q_0, ab) \\ R_6 : \quad \delta(q_0, b, b) &= \delta(q_0, bb) \\ R_7 : \quad \delta(q_0, c, Z_0) &= \delta(q_1, Z_0) \end{aligned}$$

$$R_8 : \delta(q_0, c, a) = \delta(q_1, a)$$

$$R_9 : \delta(q_0, c, b) = \delta(q_1, b)$$

$$R_{10} : \delta(q_1, a, a) = \delta(q_1, \epsilon)$$

$$R_{11} : \delta(q_1, b, b) = \delta(q_1, \epsilon)$$

$$R_{12} : \delta(q_1, \epsilon, Z_0) = \delta(q_2, Z_0)$$

We will first draw the transition graph for given δ transitions.

Now we will simulate this PDA for the input *bbacabb*.

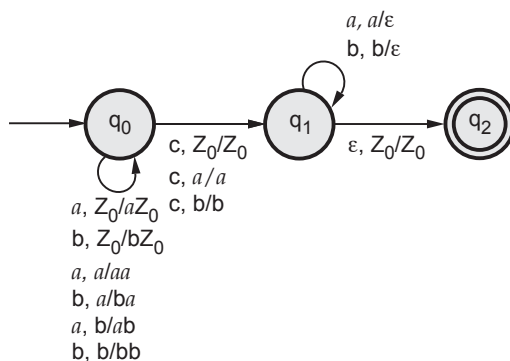


Fig. 4.2.3

From initial ID, we will try to match rules R_1 to R_{12} .

$$(q_0, \underline{b}bacabb, \underline{Z}_0) \vdash (q_0, \underline{b}acabb, \underline{bZ}_0)$$

$$\vdash (q_0, \underline{a}cabb, \underline{bbZ}_0)$$

$$\vdash (q_0, \underline{c}abb, \underline{abbZ}_0)$$

$$\vdash (q_1, \underline{a}bb, \underline{abbZ}_0)$$

$$\vdash (q_1, \underline{bb}, \underline{bbZ}_0)$$

$$\vdash (q_1, b, bZ_0)$$

$$\vdash (q_1, \epsilon, Z_0)$$

$$\vdash (q_2, Z_0)$$

which is ACCEPT state.

Thus the input *bbacabb* is accepted by given PDA.

Example 4.2.4 Design PDA to accept the language

$$L = \{w/w \in (a + b)^* \text{ and } n_a(w) = n_b(w)\}.$$

Solution : This is a language of equal number of a 's and equal number of b 's. Initially when stack is empty then whatever we read either ' a ' or ' b ' we will simply push it onto the stack. Now if we read ' a ' and at the top of the stack if ' b ' is present then it will be erased by ϵ . (i.e. we pop it) Similarly if we read ' b ' and top of the stack contains ' a ' then erase it by ϵ . The instantaneous description for this language is as given below.

$$\delta(q_0, a, Z_0) = (q_0, aZ_0)$$

$$\delta(q_0, b, Z_0) = (q_0, bZ_0)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, b) = (q_0, bb)$$

$$\delta(q_0, a, b) = (q_0, \epsilon)$$

$$\delta(q_0, b, a) = (q_0, \epsilon)$$

$$\delta(q_0, \epsilon, Z_0) = (q_1, Z_0)$$

where q_0 is a start state and q_1 is a final state. When we reach to this state with ϵ input and having an empty stack. We move to accept/final state. Let us simulate this for a string ' $aababb$ '.

$$\delta(q_0, aababb, Z_0) \vdash (q_0, ababb, aZ_0)$$

$$\vdash (q_0, babb, aaZ_0)$$

$$\vdash (q_0, abb, aZ_0)$$

$$\vdash (q_0, bb, aaZ_0)$$

$$\vdash (q_0, b, aZ_0)$$

$$\vdash (q_0, \epsilon, Z_0)$$

$$\vdash (q_1, Z_0)$$

ACCEPT state.

Example 4.2.5 Design a PDA for the language

$$L = \{w \mid w \in (a + b)^* \text{ and } n_a(w) > n_b(w)\}.$$

Solution :

• **Logic**

$n_a(w)$ means total number of a 's in input string and $n_b(w)$ means total number of b 's in input string. The problem states that total number of a 's are more than total number of b 's in input string. The logic for this PDA will be -

If we read a or b we will simply push it onto the stack. If the stack top has a symbol a and we read a , then also push it onto the stack. Same is true for b . But if we read ' a '

and stack top symbol is 'b' then pop. Also if we read 'b' and stack top symbol is 'a' then pop the stack. Finally if we read ϵ (i.e. Complete string is read) then stack should contain a on the top. This means that total number of a 's are more than total number of b 's. The ID can be,

• **Design**

$$\delta(q_0, a, Z_0) = (q_0, aZ_0)$$

$$\delta(q_0, b, Z_0) = (q_0, bZ_0)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, b) = (q_0, bb)$$

$$\delta(q_0, a, b) = (q_0, \epsilon)$$

$$\delta(q_0, b, a) = (q_0, \epsilon)$$

$$\delta(q_0, \epsilon, a) = (q_f, a)$$

where $P = (\{q_0, q_f\}, \{a, b\}, \{a, b, Z_0\}, \delta, q_0, Z_0, \{q_f\})$

Simulation :

We will take some string to simulate this PDA for some input string.

Consider the input $aababab$.

$$\delta(q_0, aababab, Z_0) \vdash (q_0, ababab, aZ_0)$$

$$\vdash (q_0, babab, aZ_0)$$

$$\vdash (q_0, abab, aZ_0)$$

$$\vdash (q_0, bab, aaZ_0)$$

$$\vdash (q_0, ab, aZ_0)$$

$$\vdash (q_0, b, aaZ_0)$$

$$\vdash (q_0, \epsilon, aZ_0)$$

$$\vdash (q_f, a)$$

Accept state.

Thus input gets accepted.

Example 4.2.6 Design PDA for the language that accepts strings with $n_a(w) < n_b(w)$ where $w \in (a + b)^*$.

Solution : This problem is similar to previous problem. The only difference is that in this problem the language requires more number of b 's than total number of a 's.

Hence the ID will be -

$$\delta(q_0, a, Z_0) = (q_0, aZ_0)$$

$$\delta(q_0, b, Z_0) = (q_0, bZ_0)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, b) = (q_0, bb)$$

$$\delta(q_0, b, a) = (q_0, \epsilon)$$

$$\delta(q_0, a, b) = (q_0, \epsilon)$$

$$\delta(q_0, \epsilon, b) = (q_f, b) \leftarrow \text{The b's are more in number.}$$

Now we will **simulate** this PDA for the input "abbab".

$$\delta(q_0, abbab, Z_0) \vdash (q_0, bbab, aZ_0)$$

$$\vdash (q_0, bab, Z_0)$$

$$\vdash (q_0, ab, bZ_0)$$

$$\vdash (q_0, b, Z_0)$$

$$\vdash (q_0, \epsilon, bZ_0)$$

$$\vdash (q_f, b)$$

Accept state.

Example 4.2.7 Design a PDA that accepts a string of well formed parenthesis. Consider the parenthesis is as (,), [,], { , }.

Solution : The problem of checking well formedness of parenthesis is similar to the problem of checking equal number of a's and equal number of b's. But always (, [, or { i.e. left parenthesis come first and then corresponding),], or } i.e. right parenthesis appear. This is what is a meaning of well - formedness. The ID can be constructed as below :

Initially i.e. in state q_0 either (, { , or [will be scanned and at that time the stack will be empty. Then we will simply push it onto stack.

$$\therefore \delta(q_0, (, Z_0) \vdash (q_1, (Z_0)$$

$$\delta(q_0, [, Z_0) \vdash (q_1, [Z_0)$$

$$\delta(q_0, \{, Z_0) \vdash (q_1, \{Z_0)$$

Further if we read more left parenthesis then we will simply push them onto the stack.

$$\delta(q_1, (, () = (q_1, (()$$

$$\delta(q_1, [, [] = (q_1, [[])$$

$$\delta(q_1, \{, \{\} = (q_1, \{\{\})$$

$$\delta(q_1, (, [] = (q_1, ([)$$

$$\delta(q_1, (, \{\} = (q_1, (\{\})$$

$$\delta(q_1, [, () = (q_1, [())$$

$$\delta(q_1, \{, () = (q_1, \{())$$

$$\delta(q_1, \{, [] = (q_1, \{[])$$

$$\delta(q_1, [, \{\} = (q_1, [, \{\})$$

Thus if we read any opening parenthesis we go on pushing it onto the stack. But as soon as we read closing parenthesis or right parenthesis we start popping the stack contents.

$$\therefore \delta(q_1,), () = (q_1, \epsilon)$$

$$\delta(q_1,], [] = (q_1, \epsilon)$$

$$\delta(q_1, \}, \{\} = (q_1, \epsilon)$$

After reading the complete input string we will get ϵ to read but at the same time the contents of stack should be removed and it should simply contain Z_0 . At this time the PDA should enter the final state and that too in state q_0 , so that all steps can be repeated if needed. This transition can be as shown -

$$\delta(q_1, \epsilon, Z_0) = (q_0, Z_0)$$

To summarize these steps we will write ID as -

$$\delta(q_0, (, Z_0) = (q_1, (Z_0)$$

$$\delta(q_0, [, Z_0) = (q_1, [Z_0)$$

$$\delta(q_0, \{, Z_0) = (q_1, \{Z_0)$$

$$\delta(q_1, (, () = (q_1, (()$$

$$\delta(q_1, [, [] = (q_1, [[])$$

$$\delta(q_1, \{, \{\} = (q_1, \{\{\})$$

$$\delta(q_1, (, [] = (q_1, ([)$$

$$\delta(q_1, (, \{\} = (q_1, (\{\})$$

$$\delta(q_1, [, () = (q_1, [())$$

$$\delta(q_1, \{, () = (q_1, \{())$$

$$\delta(q_1, \{, [] = (q_1, \{[])$$

$$\delta(q_1, [, \{) = (q_1, [\{)$$

$$\delta(q_1,), () = (q_1, \epsilon)$$

$$\delta(q_1,], [] = (q_1, \epsilon)$$

$$\delta(q_1, \}, \{) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, Z_0) = (q_0, Z_0)$$

The PDA $P = (\{q_0, q_1\}, \{(\{[, \{), \}, \}, \{[, \{, Z_0\}, \delta, q_0, Z_0, \{q_0\})$

Let us simulate it for some input string.

$((\{ \} []))$

$$\delta(q_0, ((\{ \} [])), Z_0) \vdash (q_1, (\{ \} []), (Z_0)$$

$$\vdash (q_1, \{ \} []), \{ (Z_0)$$

$$\vdash (q_1, []), (Z_0)$$

$$\vdash (q_1, []), [(Z_0)$$

$$\vdash (q_1,), (Z_0)$$

$$\vdash (q_1, \epsilon, Z_0)$$

$$(q_0, Z_0)$$

ACCEPT state.

Example 4.2.8 Construct PDA for the language

$$\{L = a^m b^m c^n \mid m, n \geq 1\}.$$

Solution : This is the language in which all the a 's are followed by equal number of b 's followed by any number of c 's. The simple logic that we can apply is : As we read go on pushing each a onto the stack. As soon as we read b , pop one a from the stack. Repeat this process while reading all b 's. Now when we read c , simply read c and do nothing, because number of c 's are arbitrary in given language. The Instantaneous Description (ID) for this logic can be as follows.

$$\delta(q_0, a, z_0) = \delta(q_0, a Z_0)$$

$$\delta(q_0, a, a) = \delta(q_0, aa)$$

$$\delta(q_0, b, a) = \delta(q_1, \epsilon)$$

$$\delta(q_1, b, a) = \delta(q_1, \epsilon)$$

$$\delta(q_1, c, Z_0) = \delta(q_1, Z_0)$$

$$\delta(q_1, \epsilon, Z_0) = \delta(q_2, Z_0)$$

Let us derive this PDA for the same example.

$$\delta(q_0, aabbccc, Z_0) \vdash \delta(q_0, abbccc, aZ_0)$$

$$\vdash \delta(q_0, bbccc, aaZ_0)$$

$$\vdash \delta(q_1, bccc, aZ_0)$$

$$\vdash \delta(q_1, ccc, Z_0)$$

$$\vdash \delta(q_1, cc, Z_0)$$

$$\vdash \delta(q_1, c, Z_0)$$

$$\vdash \delta(q_1, \epsilon, Z_0)$$

$$\vdash \delta(q_2, Z_0)$$

ACCEPT

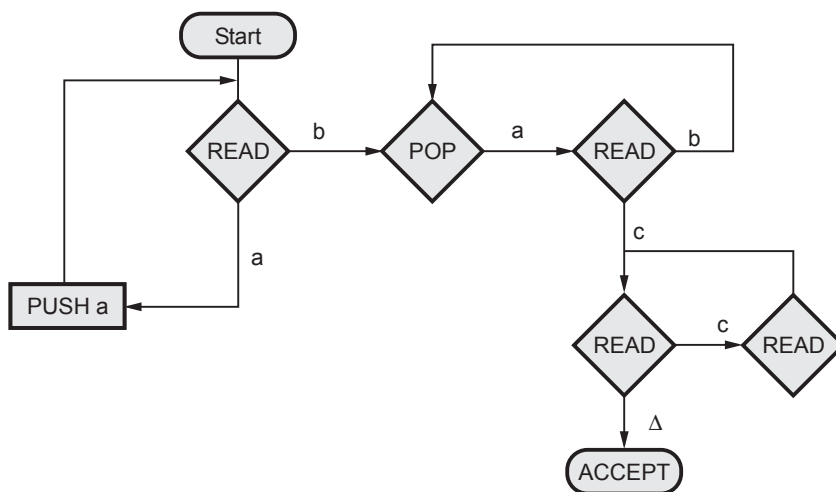


Fig. 4.2.4 PDA for $a^m b^m c^n$

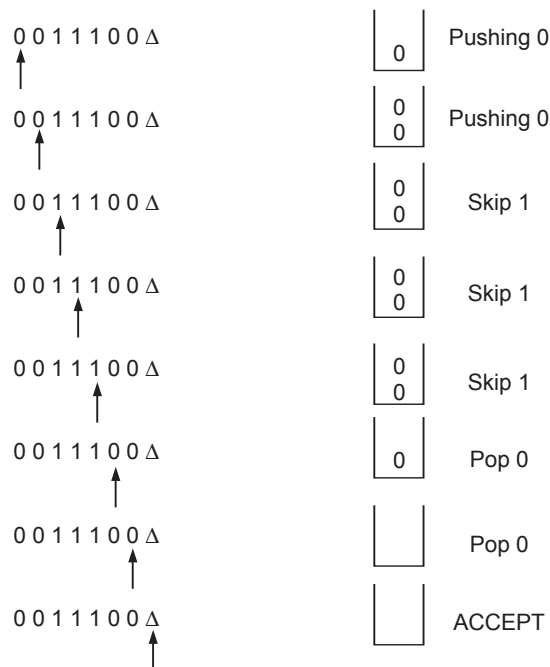
Example 4.2.9 Build a PDA for the language

$$L = \{0^n 1^m 0^n \mid m, n \geq 1\} \text{ by empty stack.}$$

SPPU : Dec.-13, Marks 7

Solution : In this PDA n number of 0's are followed by any number of 1's followed by n number of 0's. Hence the logic for design of such PDA will be as follows. Push all 0's onto the stack on encountering first zeros. Then if we read 1, just do nothing. Then read 0 and on each read of 0, pop one 0 from the stack. For instance :

This scenario can be written in the ID form as,



$$\delta(q_0, 0, Z_0) = \delta(q_0, 0 Z_0)$$

$$\delta(q_0, 0, 0) = \delta(q_0, 00)$$

$$\delta(q_0, 1, 0) = \delta(q_1, 0)$$

$$\delta(q_0, 1, 0) = \delta(q_1, 0)$$

$$\delta(q_1, 0, 0) = \delta(q_1, \epsilon)$$

$$\delta(q_0, \epsilon, Z_0) = \delta(q_2, Z_0) \leftarrow \text{ACCEPT state}$$

For example :

$$\delta(q_0, 0011100, Z_0) \vdash \delta(q_0, 011100, 0Z_0)$$

$$\vdash \delta(q_0, 11100, 00Z_0)$$

$$\vdash \delta(q_0, 1100, 00Z_0)$$

$$\vdash \delta(q_1, 100, 00Z_0)$$

$$\vdash \delta(q_1, 00, 00Z_0)$$

$$\vdash \delta(q_1, 0, 0Z_0)$$

$$\vdash \delta(q_1, \epsilon, Z_0)$$

$$\vdash \delta(q_2, Z_0)$$

ACCEPT

Example 4.2.10 Construct pushdown automata for each of the following language

- 1) The set of palindromes over alphabet $\{a, b\}$
- 2) The set of all string over alphabet $\{a, b\}$ with exactly twice many a's as b's
- 3) $\{a^i b^j c^k \mid i \neq j \text{ or } j \neq k\}$

SPPU : May-07, Marks 12, Dec.-16, End Sem. Marks 10, May-18, End Sem, Marks 6

Solution :

- 1) Refer example 4.2.3.
- 2) We will apply a simple logic to design this PDA if we read first 'a' then simply change the state from q_0 to q_1 . In state q_1 if a is read then push 'A', and for this second 'a', change the state from q_1 to q_0 again.

If we read single b when we are there in state q_0 then push B onto the stack. This state indicates that we are waiting for two a's to read out. On receiving two a's, this B will be popped.

The simple rule in the design of this PDA is : State q_1 indicates that a single 'a' is read so far now. The δ transitions for this PDA are as given below -

1. $\delta(q_0, a, Z_0) = (q_1, Z_0)$
2. $\delta(q_1, a, Z_0) = (q_0, AZ_0)$ i.e. push A on reading second a and change the state to q_0
3. $\delta(q_0, b, Z_0) = (q_0, BZ_0)$ i.e. initially simply push B on reading single B.
4. $\delta(q_1, b, Z_0) = (q_1, BZ_0)$ i.e. push B on reading b
5. $\delta(q_0, a, A) = (q_1, A)$ i.e. simply read the a and change state to q_1 .
6. $\delta(q_0, a, B) = (q_1, B)$ i.e. one b is already read and now one a is read. so keep B as it is
7. $\delta(q_0, b, B) = (q_0, BB)$ i.e. push B for reading a
8. $\delta(q_1, a, B) = (q_0, \epsilon)$ i.e. read two a's for single b.
9. $\delta(q_1, a, A) = (q_0, AA)$ i.e. push A onto the stack.
10. $\delta(q_0, b, A) = (q_0, \epsilon)$ pop A from stack.
11. $\delta(q_1, b, B) = (q_0, BB)$ push B for reading b.
12. $\delta(q_0, \epsilon, Z_0) = (q_0, \epsilon)$ **ACCEPT.**

Simulation for abbaaa

$$\begin{aligned}
\delta(q_0, \underline{a} b b a a a, \underline{Z}_0) &= (q_1, \underline{a} \underline{b} b a a a, Z_0) \quad \text{rule 1} \\
&= (q_1, \underline{a} b b a a a, B Z_0) \quad \text{rule 4} \\
&= (q_1, \underline{a} b \underline{b} a a a, B Z_0) \quad \text{rule 11} \\
&= (q_1, \underline{a} b b \underline{a} a a, \underline{B} B Z_0) \quad \text{rule 8} \\
&= (q_0, \underline{a} b b \underline{a} \underline{a} a, B Z_0) \quad \text{rule 6} \\
&= (q_1, \underline{a} b b b \underline{a}, \underline{B} Z_0) \quad \text{rule 8} \\
&= (q_0, \underline{a} b b \underline{a} a a, Z_0) \quad \text{rule 12} \\
&\text{ACCEPT.}
\end{aligned}$$

3) Let,
$$L = \{a^i b^j c^k \mid i \neq j \text{ or } j \neq k\}$$

Note that this is an or condition. If we consider the first condition i.e. $i \neq j$ then we have to focus on a's and b's only. At the same time any number of c's are allowed. Here $i > j$ or $j > i$ are the two applicable conditions.

Similarly in the second condition $j \neq k$ then we have to focus on b's and c's only. At the same time any number of a's are allowed.

Here $j > k$ or $k > j$ are the two applicable conditions.

We will first write the CFG for these conditions.

$$S \rightarrow P|Q|R|T|M$$

Let, A, B and C are the non terminals that generate a^+ , b^+ and c^+ respectively.

$$A \rightarrow a A | a$$

$$B \rightarrow b B | b$$

$$C \rightarrow c C | c$$

i) P generates $a^i b^j c^k$ such that $i > j$

$$P \rightarrow A D | A D C$$

$$D \rightarrow a D b | \epsilon \quad \text{Here } D = a^n b^n \text{ where } n \geq 0.$$

ii) Q generates $a^i b^j c^k$ such that $j > i$

$$Q \rightarrow D B | D B C$$

iii) R generates $a^i b^j c^k$ such that $j > k$

$$R \rightarrow B E | A B E$$

$$E \rightarrow b E c | \epsilon \quad \text{Here } E = b^n c^n \text{ where } n \geq 0$$

iv) M generates $a^i b^j c^k$ such that $k > j$

$$M \rightarrow E C I A E C$$

From these above given production rules we can write δ for PDA as follows.

$$(q, \varepsilon, S) = \{ (q, P), (q, Q), (q, R), (q, T), (q, M) \}$$

$$(q, \varepsilon, A) = \{ (q, aA), (q, a) \}$$

$$(q, \varepsilon, B) = \{ (q, bB), (q, b) \}$$

$$(q, \varepsilon, C) = \{ (q, cC), (q, c) \}$$

$$(q, \varepsilon, P) = \{ (q, AD), (q, ADC) \}$$

$$(q, \varepsilon, D) = \{ (q, aDb), (q, \varepsilon) \}$$

$$(q, \varepsilon, Q) = \{ (q, DB), (q, DBC) \}$$

$$(q, \varepsilon, R) = \{ (q, BE), (q, ABE) \}$$

$$(q, \varepsilon, E) = \{ (q, bEc), (q, \varepsilon) \}$$

$$(q, \varepsilon, M) = \{ (q, EC), (q, AEC) \}$$

$$(q, a, a) = \{ (q, \varepsilon) \}$$

$$(q, b, b) = \{ (q, \varepsilon) \}$$

$$(q, c, c) = \{ (q, \varepsilon) \}$$

Example 4.2.11 Design PDA for a language.

$$L = \{w(a+b)w^R \mid w \in (a+b)^* \text{ and } w^R \text{ is reverse of } w\}$$

SPPU : Dec.-14, End Sem, Marks 8

Solution : The PDA is a nondeterministic PDA. It can be represented as follows -

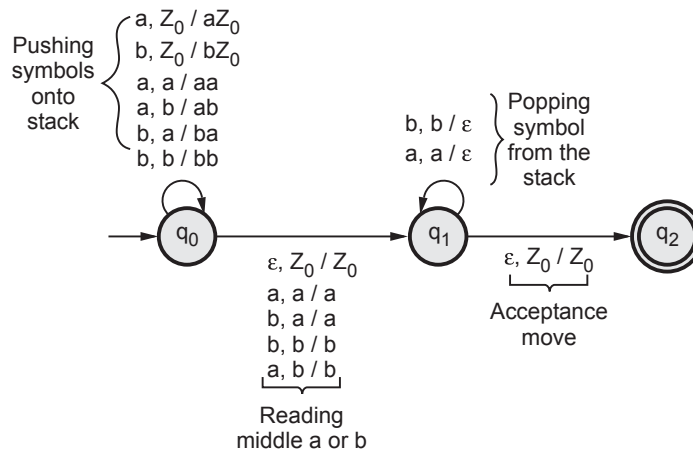


Fig. 4.2.5

Example 4.2.12 Construct transition table for PDA that accepts the languages

$L = \{a^{2n}b^n \mid n \geq 1\}$. Trace your PDA for the input with $n = 3$.

SPPU : May-15, End Sem, Marks 10

Solution : The logic for building this PDA is as follows :

Just read one a and push and change state. Then read and push next a onto the stack. Repeat this for all strings of a's. On reading each b pop one a from stack.

The δ function can be as given below.

$\delta(q_0, a, Z_0) = (q_1, Z_0)$ Simply read a and change state.

$\delta(q_1, a, Z_0) = (q_0, AZ_0)$ Push a onto the stack as A. Change state to q_0 .

$\delta(q_0, a, A) = (q_1, A)$ Simply read a and change state.

$\delta(q_1, a, A) = (q_1, AA)$ Push a as A

$\delta(q_0, b, A) = (q_0, \epsilon)$ POP A

$\delta(q_0, \epsilon, Z_0)$ ACCEPT.

Simulation for $n = 3$ i.e. aaaaaabbbb

$(q_0, aaaaaabbbb, Z_0) \vdash (q_1, aaaaabbbb, Z_0)$
 $\vdash (q_0, aaaa bbb, AZ_0)$
 $\vdash (q_1, aaabbb, AZ_0)$
 $\vdash (q_0, aa bbb, AAZ_0)$
 $\vdash (q_1, abbb, AAZ_0)$
 $\vdash (q_0, bbb, AAAZ_0)$
 $\vdash (q_0, bb, AAZ_0)$
 $\vdash (q_0, b, AZ_0)$
 $\vdash (q_0, \epsilon, Z_0)$
 $\vdash \text{ACCEPT}$

4.3 Acceptance by Final State

SPPU : Dec.-18, 19, Marks 4

The PDA accepts its input by consuming it and then enters in the final state. The formal definition is,

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be a PDA. Then the language $L(P)$ is called the **language accepted by final state**.

The language $L(P)$ can be defined as,

$$L(P) = \{w \mid (q_0, w, Z_0) \vdash^* (p, \epsilon, \alpha)\}$$

i.e. the PDA P reads the entire input w and enters in a final state p .

4.4 Acceptance by Empty Stack

On reading the input string from initial configuration for some PDA, the stack of PDA gets empty. The formal definition is -

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be the PDA then the **language accepted by empty stack $N(P)$** is defined as

$$N(P) = \{w \mid (q_0, w, Z_0) \vdash^* (p, \epsilon, \epsilon)\}$$

Review Question

1. Explain acceptance by PDA - i) By final state ii) By empty state.

SPPU : Dec.-18, 19, End Sem, Marks 4

4.5 Equivalence of Acceptance by Final State and Empty Stack

SPPU : Dec.-09,13, Marks 18

1. If $L = N(P_1)$ for some PDA P_1 then there is a PDA P_2 such that $L = L(P_2)$ That means the language accepted by empty stack PDA will also be accepted by final state PDA.
2. If there is a language $L = L(P_1)$ for some PDA P_1 then there is a PDA P_2 such that $L = N(P_2)$ That means language accepted by final state PDA is also acceptable by empty stack PDA.

Theorem 1 : The language accepted by empty stack PDA is also acceptable by final state PDA.

Proof : Let $P_1 = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \phi)$ be a PDA, accepted by empty stack.

Now, construct a PDA P_2 using P_1 .

The construction will be -

$$P_2 = (Q \cup \{q'_0, q_f\}, \Sigma, \Gamma \cup \{X_0\}, \delta', q'_0, X_0, \{q_f\})$$

Where, the mapping function is given by -

Rule 1) $\delta'(q'_0, \epsilon, X_0) = \{(q_0, Z_0, X_0)\}$

Rule 2) $\delta'(q, a, Z) = \delta(q, a, z)$ for all q in Q and a in $\Sigma \cup \{\epsilon\}$ and Z in Γ .

Rule 3) $\delta'(q, \epsilon, X_0) = \{(q_f, \epsilon)\}$ for all q in Q .

Thus PDA P_2 first enters into the initial configuration of P_1 without using any input symbol (i.e. ϵ transition). Then using Rule 2 the simulation of P_1 is carried out. When P_1 has an empty stack then P_2 enters in final state q_f (Refer Rule 3).

This shows that for every PDA accepting by empty stack there exists equivalent PDA by final state.

Theorem 2 : The language accepted by final state PDA is also acceptable by empty stack PDA.

Proof : Let, $P_1 = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be the PDA which accepts the language by empty stack.

Now, we will construct PDA P_2 using P_1 . The construction will be -

$$P_2 = (Q \cup \{q'_0, q_f\}, \Sigma, \Gamma \cup \{X_0\}, \delta', q'_0, X_0, \phi)$$

Where,

Rule 1) $\delta'(q'_0, \epsilon, X_0) = \{q_0, Z_0\}$

Rule 2) $\delta'(q, a, Z) = \delta(q, a, Z)$ for all q in Q , a in Σ and Z in Γ .

Rule 3) $\delta'(q, \epsilon, Z) = \delta(q, \epsilon, Z) \cup \{(d, \epsilon)\}$ for all $Z \in \Gamma \cup X_0$ and $q \in F$ and d is a new dead state.

Rule 4) $\delta'(d, \epsilon, Z) = \{d, \epsilon\}$ for all $Z \in \Gamma \cup X_0$.

Thus PDA P_2 enters into initial configuration of P_1 without consuming any input symbols. When some symbol is read then using Rule 2 the PDA P_1 is simulated and it reaches to final state of PDA P_1 . The PDA P_2 at this stage simply makes a guess about the input string whether it is terminated or not.

If input string is not terminated, P_2 again simulates P_1 , otherwise P_2 enters the dead state d (Rule 3). Finally using Rule 4 using ϵ move,

P_2 erases all symbols and empties the stack. This shows that for every PDA accepting by final state there exists equivalent PDA by empty stack.

Example 4.5.1 Let $L = \{a^i b^j c^k \mid i, j, k \geq 0 \text{ and } i + j = k\}$

a) Find a PDA (which accepts via final state) that recognizes L

b) Find a PDA (which accepts via empty stack) that recognizes L .

SPPU : Dec.-09, Marks 18

Solution : Let, $L = \{a^i b^j c^k \mid i + j = k\}$ be a given language. The simple logic for designing this PDA will be - just read a's and push A for each corresponding a. Then read b's and push A for each corresponding b. Now on reading each c just pop a single A from stack.

a) Acceptance via final state.

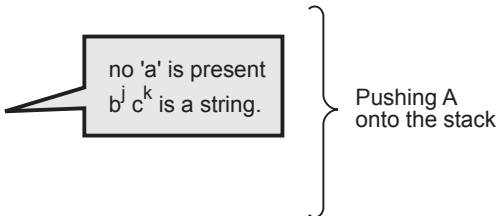
$$\delta(q_0, a, Z_0) = (q_1, AZ_0)$$

$$\delta(q_1, a, A) = (q_1, AA)$$

$$\delta(q_0, b, Z_0) = (q_2, AZ_0)$$

$$\delta(q_1, b, A) = (q_2, AA)$$

$$\delta(q_2, b, A) = (q_2, AA)$$



$$\left. \begin{array}{l} \delta(q_2, c, A) = (q_3, \epsilon) \\ \delta(q_3, c, A) = (q_3, \epsilon) \end{array} \right\} \text{popping A from the stack.}$$

$\delta(q_3, \epsilon, Z_0) = (q_3, \epsilon) \rightarrow$ Acceptance by final state q_3 .

b) Acceptance via empty stack.

$$\delta(q_0, a, Z_0) = (q_1, A Z_0)$$

$$\delta(q_1, a, A) = (q_1, A A)$$

$$\delta(q_0, b, Z_0) = (q_2, A Z_0)$$

$$\delta(q_1, b, A) = (q_2, A A)$$

$$\delta(q_2, b, A) = (q_2, A A)$$

$$\delta(q_2, c, A) = (q_3, \epsilon)$$

$$\delta(q_3, c, A) = (q_3, \epsilon)$$

$\delta(q_3, \epsilon, Z_0) = (q_0, \epsilon) \rightarrow$ Acceptance by empty stack.

Review Question

1. Give formal definition of -
 - i) Pushdown Automata (PDA)
 - ii) Acceptance of PDA in terms of final state
 - iii) Acceptance of PDA in terms of null store
 - iv) Instantaneous description of PDA.

SPPU : Dec.-13, Marks 6

4.6 Non-deterministic PDA (NPDA)

SPPU : Dec.-17, May-18, Marks 6

The Non deterministic PDA (NPDA) is a 7 tuple $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ where Q is a finite set of states, Σ is input set, Γ is a set of stack symbols. The δ is a mapping function. q_0 is a initial state, Z_0 is a stack symbol and F is set of finite states A NPDA is like NFA, except it has stack. There can be more than one moves possible on reading the input symbol.

NPDA is strictly more powerful than deterministic PDA.

Example 4.6.1 Construct NPDA that accepts the language generated by $S = S + S \mid S * S \mid 4$.

SPPU : Dec.-17, End Sem, Marks 6

Solution : The PDA can be,

$$A = \{ \{q\}, \{*, 4, +\}, \{S, 4, +, *\}, \delta, q, S, \phi \}$$

The δ can be

$$R1 : \delta(q, \epsilon, S) = \{(q, S + S), (q, S * S), (q, 4)\}$$

$$R2 : \delta(q, \epsilon, 4) = \{(q, \epsilon)\}$$

$$R3 : \delta(q, \epsilon, +) = \{(q, \epsilon)\}$$

$$R4 : \delta(q, \epsilon, *) = \{(q, \epsilon)\}$$

Thus the generated PDA is NPDA because there are more than one moves for same input symbol and from same current state.

Example 4.6.2 Construct a NPDA that accepts the language $L = \{a^{2n} \mid n > 0\}$

SPPU : May-18, End Sem, Marks 6

Solution : The ID for PDA is,

$$\left. \begin{array}{l} \delta(q_0, a, Z_0) = (q_1, aZ_0) \\ \delta(q_1, a, a) = (q_1, aa) \end{array} \right\} \text{Push}$$

$$\left. \begin{array}{l} \delta(q_1, a, a) = (q_2, \epsilon) \\ \delta(q_2, a, a) = (q_3, \epsilon) \\ \delta(q_2, \epsilon, Z_0) = (q_3, \epsilon) \end{array} \right\} \text{POP}$$

$$\delta(q_3, \epsilon, Z_0) = (q_3, \epsilon) \quad \text{ACCEPT}$$

Review Question

1. Explain closure property of CFL with suitable example.

SPPU : Dec.-17, End Sem, Marks 6

4.7 PDA and Context Free Language

SPPU : Dec.-05,08,11,14,16,17,18,19, May-14,15,16,18, Marks 12

4.7.1 PDA to CFG

Algorithm for getting production rules of CFG

1. If q_0 is start state in PDA and q_n is final state of PDA then $[q_0 Z q_n]$ becomes **start state** of CFG. Here Z represents the stack symbol.

2. The production rule for the ID of the form $\delta(q_i, a, Z_0) = (q_{i+1}, Z_1 Z_2)$ can be obtained as

$$\delta(q_i, Z_0, q_{i+k}) \rightarrow a(q_i, Z_1, q_m)(q_m, Z_2, q_{i+k})$$

where q_i, q_m represents the intermediate states, Z_0, Z_1, Z_2 are stack symbols and a is input symbol.

3. The production rule for the ID of the form

$$\delta(q_i, a, Z_0) = (q_{i+1}, \epsilon) \text{ can be converted as } (q_i, Z_0, q_{i+1}) \rightarrow a$$

Example 4.7.1 Consider the PDA with following moves.

$$\delta(q_0, a, Z_0) = \{(q_0, a, Z_0)\}$$

$$\delta(q_0, a, a) = \{(q_0, a, a)\}$$

$$\delta(q_0, b, a) = \{(q_1, e)\}$$

$$\delta(q_1, b, a) = \{(q_1, e)\}$$

$$\delta(q_0, e, Z_0) = \{(q_1, e)\}$$

Obtain CFG equivalent to PDA

SPPU : Dec.-08, Marks 8, May-14, Marks 12, Dec.-19, End Sem, Marks 6

Solution : 1) We will create production rules for start symbol, S. These are as follow :

$$S \rightarrow (q_0, Z_0, q_0)$$

$$S \rightarrow (q_0, Z_0, q_1)$$

- 2) Consider the rule $\delta(q_0, a, Z_0) = (q_0, aZ_0)$

$$\text{i) } (q_0, Z_0, q_0) \rightarrow a(q_0, a, q_0)(q_0, Z_0, q_0)$$

$$\text{ii) } (q_0, Z_0, q_0) \rightarrow a(q_0, a, q_1)(q_1, Z_0, q_0)$$

$$\text{iii) } (q_0, Z_0, q_1) \rightarrow a(q_0, a, q_0)(q_0, Z_0, q_1)$$

$$\text{iv) } (q_0, Z_0, q_1) \rightarrow a(q_0, a, q_1)(q_1, Z_0, q_1)$$

- 3) Consider the rule $\delta(q_0, a, a) = (q_0, a, a)$

$$\text{v) } (q_0, a, q_0) \rightarrow a(q_0, a, q_0)(q_0, a, q_0)$$

$$\text{vi) } (q_0, a, q_0) \rightarrow a(q_0, a, q_1)(q_1, a, q_0)$$

$$\text{vii) } (q_0, a, q_1) \rightarrow a(q_0, a, q_0)(q_0, a, q_1)$$

$$\text{viii) } (q_0, a, q_1) \rightarrow a(q_0, a, q_1)(q_1, a, q_1)$$

- 4) Consider the rule $\delta(q_0, b, a) = (q_1, \epsilon)$

$$\text{ix) } (q_0, a, q_1) \rightarrow b$$

- 5) Consider the rule $\delta(q_1, b, a) = (q_1, \epsilon)$

$$\text{x) } (q_1, a, q_1) \rightarrow b$$

6) Consider the rule $(q_0, \varepsilon, Z_0) = (q_1, \varepsilon)$

xi) $(q_0, Z_0, q_1) \rightarrow \varepsilon$

If we assume the state names as follows -

$(q_0, Z_0, q_0) = A$		$S \rightarrow A \mid B$
$(q_0, Z_0, q_1) = B$	Above obtained	$A \rightarrow aCA \mid aDE$
$(q_0, a, q_0) = C$	$\xrightarrow[\text{rules}]{\text{Conversion}}$	$B \rightarrow aCB \mid aDF$
$(q_1, a, q_1) = D$		$C \rightarrow aCC \mid aDG$
$(q_1, Z_0, q_0) = E$		$D \rightarrow aCD \mid aDH$
$(q_1, Z_0, q_1) = F$		$D \rightarrow b$
$(q_1, a, q_0) = G$		$H \rightarrow b$
$(q_1, a, q_1) = H$		$B \rightarrow \varepsilon$

The rules for E, F, G are missing. Hence we will eliminate production rules on RHS containing E, F or G non terminals. Finally the production rules for the equivalent grammar are

$S \rightarrow A \mid B$
 $A \rightarrow aCA$
 $B \rightarrow aCB \mid \varepsilon$
 $C \rightarrow aCC$
 $D \rightarrow aCD \mid aDH \mid b$
 $H \rightarrow b$

Example 4.7.2 Obtain CFG for the PDA given below

$\delta(q_0, l, Z_0) = \{q_0, XZ_0\}$

$\delta(q_0, l, X) = (q_0, XX)$

$\delta(q_0, 0, X) = (q_1, X)$

$\delta(q_0, e, Z_0) = (q_0, e)$

$\delta(q_1, l, X) = (q_1, e)$

$\delta(q_0, l, Z_0) = (q_0, Z_0)$

SPPU : May-15,16, End Sem, Dec.-16, End Sem. Marks 9

Solution :

1) We will construct production rules for start symbol S.

$S \rightarrow (q_0, Z_0, q_0)$

$S \rightarrow (q_0, Z_0, q_1)$

2) Consider the rule $\delta(q_0, 1, Z_0) = (q_0, XZ_0)$

i) $(q_0, Z_0, q_0) \rightarrow 1(q_0, X, q_0)(q_0, Z_0, q_0)$

ii) $(q_0, Z_0, q_0) \rightarrow 1(q_0, X, q_0)(q_0, Z_0, q_0)$

iii) $(q_0, Z_0, q_1) \rightarrow 1(q_0, X, q_0)(q_0, Z_0, q_1)$

iv) $(q_0, Z_0, q_1) \rightarrow 1(q_0, X, q_1)(q_1, Z_0, q_1)$

3) Consider the rule $\delta(q_0, 1, X) = (q_0, XX)$

v) $(q_0, X, q_0) \rightarrow 1(q_0, X, q_0)(q_0, X, q_0)$

vi) $(q_0, X, q_0) \rightarrow 1(q_0, X, q_1)(q_1, X, q_0)$

vii) $(q_0, X, q_1) \rightarrow 1(q_0, X, q_0)(q_0, X, q_1)$

viii) $(q_0, X, q_1) \rightarrow 1(q_0, X, q_1)(q_1, X, q_1)$

4) Consider the rule $\delta(q_0, 0, X) = (q_1, X)$

ix) $(q_0, X, q_0) \rightarrow 0(q_0, X, q_0)$

x) $(q_0, X, q_1) \rightarrow 0(q_0, X, q_1)$

5) Consider the rule $\delta(q_0, \epsilon, Z_0) = (q_0, \epsilon)$

xi) $(q_0, Z_0, q_0) \rightarrow \epsilon$

6) Consider the rule $\delta(q_1, 1, X) = (q_1, \epsilon)$

xii) $(q_1, X, q_1) \rightarrow 1$

7) Consider the rule $\delta(q_0, 1, Z_0) = (q_0, Z_0)$

xiii) $(q_0, Z_0, q_0) \rightarrow 1(q_0, Z_0, q_0)$

xiv) $(q_0, Z_0, q_1) \rightarrow 1(q_0, Z_0, q_1)$

If we assume following state names for above generated 1 to 7 production rules then

$S \rightarrow A \mid B$

$A \rightarrow 1 \text{ CA} \mid 1 \text{ CE}$

$B \rightarrow 1 \text{ CB} \mid 1 \text{ DF}$

$C \rightarrow 1 \text{ CC} \mid 1 \text{ DG}$

$D \rightarrow 1 \text{ CD} \mid 1 \text{ DH}$

$C \rightarrow 0C$

$D \rightarrow 0D$

$A \rightarrow \epsilon$

$H \rightarrow 1$

$A \rightarrow 1A$

$B \rightarrow 1B$

$(q_0, Z_0, q_0) = A$

$(q_0, Z_0, q_1) = B$

$(q_0, X, q_0) = C$

$(q_0, X, q_1) = D$

$(q_1, Z_0, q_0) = E$

$(q_1, Z_0, q_1) = F$

$(q_1, X, q_0) = G$

$(q_1, X, q_1) = H$

$\xrightarrow{\text{CFG rules}}$

The production rules for E, F and G are not defined. Hence we will eliminate the production rules on RHS containing the non terminals E, F and G. The resultant CFG rules will be,

$$S \rightarrow A \mid B$$

$$A \rightarrow 1 CA \mid 1A \mid \epsilon$$

$$B \rightarrow 1 CB \mid 1 B$$

$$C \rightarrow 1 CC \mid 0 C$$

$$D \rightarrow 1 CD \mid 1 DH \mid 0D$$

$$H \rightarrow 1$$

The rules for B, C and D does not lead to any terminal symbols (i.e. the rules generate continuous non terminals). Hence we will eliminate them. Finally we will get -

$$S \rightarrow A \Rightarrow S \rightarrow 1 S \mid \epsilon$$

$$A \rightarrow 1 A \mid \epsilon$$

Example 4.7.3 Find the grammar G equivalent to the following PDA.

$M = (\{q_0, q_1\}, \{0,1\}, \{X, Z_0\}, q_0, Z_0, d, f)$ and δ is

i) $\delta(q_0, 0, Z_0) = (q_0, XZ_0)$ ii) $\delta(q_0, 0, X) = (q_0, XX)$

iii) $\delta(q_0, 1, X) = (q_1, \Delta)$ iv) $\delta(q_1, 1, X) = (q_1, \Delta)$

v) $\delta(q_1, \Delta, X) = (q_1, \Delta)$ vi) $\delta(q_1, \Delta, Z_0) = (q_1, \Delta)$

SPPU : Dec.-14, End Sem, Marks 9

Solution : The corresponding grammar G can be constructed as follows,

- 1) The production rule for start symbol S is

$$S \rightarrow (q_0, Z_0, q_0)$$

$$S \rightarrow (q_0, Z_0, q_1)$$

- 2) Consider the rule $\delta(q_0, 0, Z_0) = (q_0, XZ_0)$

$$i) (q_0, Z_0, q_0) \rightarrow 0(q_0, X, q_0)(q_0, Z_0, q_0)$$

$$ii) (q_0, Z_0, q_0) \rightarrow 0(q_0, X, q_1)(q_1, Z_0, q_0)$$

$$iii) (q_0, Z_0, q_1) \rightarrow 0(q_0, X, q_0)(q_0, Z_0, q_1)$$

$$iv) (q_0, Z_0, q_1) \rightarrow 0(q_0, X, q_1)(q_1, Z_0, q_1)$$

- 3) Consider the rule $\delta(q_0, 0, X) = (q_0, XX)$

$$v) (q_0, X, q_0) \rightarrow 0(q_0, X, q_0)(q_0, X, q_0)$$

$$vi) (q_0, X, q_0) \rightarrow 0(q_0, X, q_1)(q_1, X, q_0)$$

- vii) $(q_0, X, q_1) \rightarrow 0 (q_0, X, q_0) (q_0, X, q_1)$
 viii) $(q_0, X, q_1) \rightarrow 0 (q_0, X, q_1) (q_1, X, q_1)$
 4) Consider the rule $\delta(q_0, 1, X) = (q_1, \Delta)$
 ix) $(q_0, X, q_1) \rightarrow 1$
 5) Consider the rule $\delta(q_1, 1, X) = (q_1, \Delta)$
 x) $(q_1, X, q_1) \rightarrow 1$
 6) Consider the rule $\delta(q_1, \Delta, X) = (q_1, \Delta)$
 xi) $(q_1, X, q_1) \rightarrow \varepsilon$
 7) Consider the rule $\delta(q_1, \Delta, Z_0) = (q_1, \Delta)$
 xii) $(q_1, Z_0, q_1) \rightarrow \varepsilon$

If we assume

$(q_1, Z_0, q_0) = A$	$S \rightarrow A \mid B$
$(q_0, Z_0, q_1) = B$	$A \rightarrow 0CA \mid 0DE$
$(q_0, X, q_0) = C$	$B \rightarrow 0CB \mid 0DG$
$(q_0, X, q_1) = D$	$\xrightarrow[\text{CFG will be}]{\text{The}} C \rightarrow 0CC \mid 0DF$
$(q_1, Z_0, q_0) = E$	$D \rightarrow 0CD \mid 0DH$
$(q_1, X, q_0) = F$	$D \rightarrow 1$
$(q_1, Z_0, q_1) = G$	$H \rightarrow 1$
$(q_1, X, q_1) = H$	$H \rightarrow \varepsilon$
	$G \rightarrow \varepsilon$

There is no rule for E and F. Hence

We will eliminate RHS rules containing E and F. Hence the final production rules are,

$$\begin{aligned}
 S &\rightarrow A \mid B \\
 A &\rightarrow 0CA \\
 B &\rightarrow 0CB \mid 0DG \\
 C &\rightarrow 0CC \\
 D &\rightarrow 0CD \mid 0DH \mid 1 \\
 H &\rightarrow 1 \mid \varepsilon \\
 G &\rightarrow \varepsilon
 \end{aligned}$$

The rule for A, C generate infinite sequence of non terminals. Hence we will eliminate these non terminals. Finally we will get simplified rules as,

$$S \rightarrow B$$

$$B \rightarrow 0DG$$

$$D \rightarrow 0DH \mid 1$$

$$H \rightarrow 1 \mid \varepsilon$$

$$G \rightarrow \varepsilon$$

Example 4.7.4 Construct a PDA accepting the language

$L = \{a^n b^m a^n \mid n, m \geq 1\}$ by null store. Then convert this PDA to CFG.

SPPU : Dec.-11, Marks 8+10, Dec.-19, End Sem, Marks 6

Solution : The instantaneous description for above problem is,

$$\delta(q_0, a, Z_0) = \delta(q_0, a, Z_0)$$

$$\delta(q_0, a, a) = \delta(q_0, aa)$$

$$\delta(q_0, b, a) = \delta(q_1, a)$$

$$\delta(q_1, b, a) = \delta(q_1, a)$$

$$\delta(q_1, a, a) = \delta(q_1, \varepsilon)$$

$$\delta(q_1, \varepsilon, Z_0) = \delta(q_2, \varepsilon)$$

The conversion of above PDA to CFG will be -

The corresponding grammar can be constructed as follows.

Let $G = (N, T, P, S)$ where

$$N = \{S, [q_0, a, q_0], [q_0, Z_0, q_0], [q_0, a, q_1], [q_0, Z_0, q_1], [q_1, a, q_1], [q_1, Z_0, q_1], [q_1, a, q_0] \text{ and } [q_1, Z_0, q_0]\}$$

$$T = \{a, b\}$$

The productions P are constructed as follows.

The s-productions and

$$1) S \rightarrow [q_0, Z_0, q_0]$$

$$2) S \rightarrow [q_0, Z_0, q_1] \text{ are the two transitions from } q_0.$$

- 3) For $\delta [q_0, a, Z_0] = [q_0, aZ_0]$ we get
- $$[q_0, Z_0, q_0] \rightarrow a [q_0, a, q_0] [q_0, Z_0, q_0]$$
- $$[q_0, Z_0, q_0] \rightarrow a [q_0, a, q_1] [q_1, Z_0, q_0]$$
- $$[q_0, Z_0, q_1] \rightarrow a [q_0, a, q_0] [q_0, Z_0, q_1]$$
- $$[q_0, Z_0, q_1] \rightarrow a [q_0, a, q_1] [q_1, Z_0, q_1]$$
- 4) Using rule $\delta (q_0, a, a) = \{(q_0, aa)\}$ we get,
- $$[q_0, a, q_0] \rightarrow a [q_0, a, q_0] [q_0, a, q_0]$$
- $$[q_0, a, q_0] \rightarrow a [q_0, a, q_1] [q_1, a, q_0]$$
- $$[q_0, a, q_1] \rightarrow a [q_0, a, q_0] [q_0, a, q_1]$$
- $$[q_0, a, q_1] \rightarrow a [q_0, a, q_1] [q_1, a, q_1]$$
- 5) Using $\delta (q_0, b, a) = \{(q_1, a)\}$ we get,
- $$[q_0, a, q_0] \rightarrow b [q_1, a, q_0]$$
- $$[q_0, a, q_1] \rightarrow b [q_1, a, q_1]$$
- 6) $\delta (q_1, b, a) = \{(q_1, a)\}$ we get,
- $$[q_1, a, q_0] \rightarrow b [q_1, a, q_0]$$
- $$[q_1, a, q_1] \rightarrow b [q_1, a, q_1]$$
- 7) $\delta (q_1, a, a) = \{(q_1, \epsilon)\}$ gives us
- $$[q_1, a, q_1] \rightarrow a$$
- 8) $\delta (q_1, \epsilon, Z_0) = \{(q_1, \epsilon)\}$ will give
- $$[q_1, Z_0, q_1] \rightarrow \epsilon$$

Thus the required CFG is constructed.

4.7.2 CFG to PDA

For converting given CFG to PDA, by this method the necessary condition is that the first symbol on R.H.S. production must be a terminal symbol. The rule that can be used to obtain PDA from CFG is,

Rule 1 : For non terminal symbols, add following rule

$$\delta(q, \epsilon, A) = (q, \alpha)$$

where the production rule is $A \rightarrow \alpha$.

Rule 2 : For each terminal symbols, add following rule -

$$\delta(q, a, a) = (q, \epsilon) \text{ for every terminal symbol.}$$

Example 4.7.5 Construct PDA for the given CFG

$$S \rightarrow 0BB$$

$$B \rightarrow 0S \mid 1S \mid 0.$$

Test whether 010^4 is acceptable by this PDA.

Solution : Let PDA $A = \{ \{q\}, \{0,1\}, \{S,B,0,1\}, \delta, q, S, \phi \}$

The production rules δ can be given as -

$$R1 : \delta(q, \epsilon, S) = \{ (q, 0BB) \}$$

$$R2 : \delta(q, \epsilon, B) = \{ (q, 0S), (q, 1S), (q, 0) \}$$

$$R3 : \delta(q, 0, 0) = \{ (q, \epsilon) \}$$

$$R4 : \delta(q, 1, 1) = \{ (q, \epsilon) \}$$

Testing 010^4 i.e. 010000 against PDA

$$\begin{aligned} \delta(q, 010000, S) &\vdash \delta(q, 010000, 0BB) \because R1 \\ &\vdash \delta(q, 10000, BB) \because R3 \\ &\vdash \delta(q, 10000, 1SB) \because R2 \\ &\vdash \delta(q, 0000, SB) \because R4 \\ &\vdash \delta(q, 0000, 0BBB) \because R1 \\ &\vdash \delta(q, 000, BBB) \because R3 \\ &\vdash \delta(q, 000, 0BB) \because R2 \\ &\vdash \delta(q, 00, BB) \because R3 \\ &\vdash \delta(q, 00, 0B) \because R2 \\ &\vdash \delta(q, 0, B) \because R3 \\ &\vdash \delta(q, 0, 0) \because R2 \\ &\vdash \delta(q, \epsilon) \because R3 \\ &\text{ACCEPT.} \end{aligned}$$

Thus 010^4 is accepted by the PDA.

Example 4.7.6 Convert the grammar

$$S \rightarrow 0 S 1 \mid A$$

$$A \rightarrow 1 A 0 \mid S \mid \epsilon$$

to the PDA that accepts the same language by empty stack.

SPPU : Dec.-05, Marks 8, Dec.-18, End Sem, Marks 6

Solution : The PDA can be

$$A = \{ \{q\}, \{0,1\}, \{S,A,0,1\}, \delta, q, S, \phi \}$$

The δ can be,

$$R1 : \delta(q, \epsilon, S) = \{(q, 0S1), (q, A)\}$$

$$R2 : \delta(q, \epsilon, A) = \{(q, 1A0), (q, S), (q, \epsilon)\}$$

$$R3 : \delta(q, 0, 0) = \{(q, \epsilon)\}$$

$$R4 : \delta(q, 1, 1) = \{(q, \epsilon)\}$$

Example 4.7.7 Design a PDA for the following grammar :

$$S \rightarrow aSb \mid bSa \mid SS \mid \Lambda$$

Is the resultant PDA deterministic or non-deterministic ? Justify your answer.

SPPU : May-14, Marks 6

Solution : The PDA can be,

$$A = \{q\}, \{a, b\}, \{S, a, b\}, \delta, q, S, \phi\}$$

The δ can be -

$$R1 : \delta(q, \epsilon, S) = \{(q, aSb), (q, bSa), (q, SS), (q, \epsilon)\}$$

$$R2 : \delta(q, a, a) = \{(q, \epsilon)\}$$

$$R3 : \delta(q, b, b) = \{(q, \epsilon)\}$$

The generated PDA is nondeterministic PDA because there can be more than one different moves for the same input symbol and from same current state.

Example 4.7.8 Construct PDA that accepts language generated by following CFG :

$$S \rightarrow SS \mid (S) \mid ()$$

SPPU : Dec.-17, End Sem, Marks 6

Solution : The δ for given CFG for constructing PDA is as follows,

$$\delta(q, \epsilon, S) = \{(q, SS), (q, (S)), (q, ())\}$$

$$\delta(q, (, () = \{(q, \epsilon)\}$$

$$\delta(q,),) = \{(q, \epsilon)\}$$

Example 4.7.9 Construct a PDA that recognizes the language accepted by following DFA.

SPPU : May-18, End Sem, Marks 6

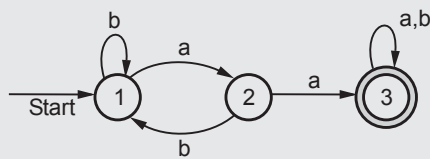


Fig. 4.7.1

Solution : • We will write r.e. for given

DFA r.e. = $b^* a(ba)^* a(a+b)^*$

- Now we will write production rules for obtained r.e.

$$S \rightarrow XaYaZ$$

$$X \rightarrow bX \mid \epsilon$$

$$Y \rightarrow baY \mid \epsilon$$

$$Z \rightarrow aZ \mid bZ \mid \epsilon$$

- The PDA can be,

$$\delta(q, \epsilon, S) = \{(q, XaYaZ)\}$$

$$\delta(q, \epsilon, X) = \{(q, bX), (q, \epsilon)\}$$

$$\delta(q, \epsilon, Y) = \{(q, baY), (q, \epsilon)\}$$

$$\delta(q, \epsilon, Z) = \{(q, aZ), (q, bZ), (q, \epsilon)\}$$

4.8 Deterministic CFLs

- The Deterministic Context-Free Grammars (DCFGs) are a proper subset of the context-free grammars.
- They are the subset of context-free grammars that can be derived from deterministic pushdown automata, and they generate the deterministic context-free languages.
- DCFGs are always unambiguous.

Theorem 1 : Prove that if L is a regular language, then $L = L(P)$ for some DPDA P .

Proof :

- Fig. 4.8.1 shows the class of various languages.

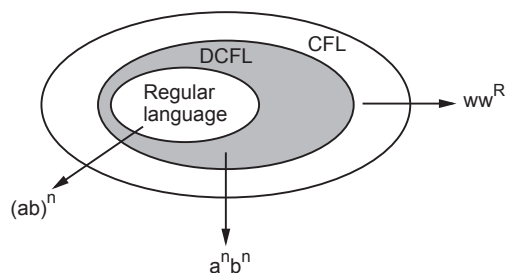


Fig. 4.8.1 Class of various languages

- A language L is a deterministic context free language (**DCFL**) if it is accepted by **DPDA**. Any language L is a subset of DCFL. For example $L = a^*b^*$ is a regular language and $L = a^n b^n$ is a DCFL, and it is accepted by DPDA. And therefore $L = a^*b^*$ is also acceptable by DPDA. Every regular language is accepted by DFA. And every DFA is a DPDA without stack. Hence it is proved that for a regular language $L = L(P)$ there exists some DPDA P .

Theorem 2 : If L is a CFL and R is a regular language then $L \cap R$ is a CFL.

Proof :

- For proving this theorem we will first create a PDA such that it is made up of two machines PDA and a FA. These two models will run parallel by **ANDing** together. The DFA will accept a language $L(R)$ and PDA will accept language L_{CFL} . Hence we can define these two models as -

$$M = (Q_{FA}, \Sigma, \delta_{FA}, q_{FA}, F_{FA}) \quad \text{be FA}$$

$$P = (Q_{PDA}, \Sigma, \Gamma, \delta_{PDA}, q_{PDA}, Z_0, F_{PDA}) \quad \text{be PDA}$$

Then the resultant PDA can be,

$$P_{New} = (Q_{PDA} \times Q_{FA}, \Sigma, \Gamma, \delta_{PM}, (q_{PDA}, q_{FA}), Z_0, F_{FA} \times F_{PDA})$$

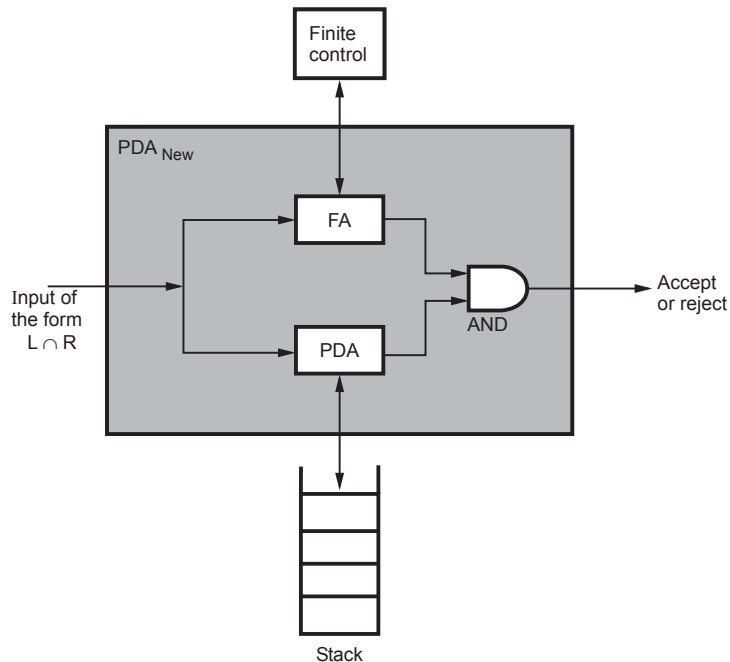


Fig. 4.8.2 $L \cap R$ is acceptable by PDA new

where δ is a mapping function it can be defined as -

$\delta((q_1, q_2), a, X)$ is defined as the set of all pairs $((q_3, q_4), Y)$ such that

1. $q_4 = \delta'_{FA}(q_2, a)$ and
2. (q_3, Y) is in $\delta_{PDA}(q_1, a, X)$

Thus for each move of PDA P , we will get the same move in PDA_{New} .

Thus the moves made by the new PDA will be,

$$((q_{PDA}, q_{FA}), W, Z_0) \xrightarrow{P_{New}^*} ((q_1, q_2), \epsilon, Y)$$

Where q_2 will act as q_f . This leads to an accepting configuration.

4.9 Fill in the Blanks

- Q.1** ____ is a graphical representation used for modeling the context free grammar.
- Q.2** ____ and ____ are types of halting states in push down automata.
- Q.3** A pushdown automata can be defined as : $(Q, \Sigma, G, q_0, z_0, A, d)$. The z_0 represents ____.
- Q.4** For every CFL, G , there exists a PDA M such that $L(G) = L(M)$ and vice versa. (True/False)
- Q.5** The transition in push down automata depends upon ____ data structure.
- Q.6** The PDA accepts a string when ____ is empty.

4.10 Multiple Choice Questions

- Q.1** The idea of automation with stack is executed in ____.
- ☐ a finite automata ☐ b push down automata
- ☐ c none of these
- Q.2** PDA is more powerful than ____.
- ☐ a turing machine ☐ b finite automata
- ☐ c both a and b
- Q.3** The instantaneous description is PDA shows ____.
- ☐ a present state ☐ b stack symbol
- ☐ c string to be processed ☐ d all of these

Q.4 If the PDA does not stop on an accepting state and the stack is not empty, the string is ____ .

- ☐ a rejected ☐ b goes into loop forever
☐ c both (a) and (b) ☐ d none of the mentioned

Q.5 Push down machine represents ____ .

- ☐ a Type-0 grammar ☐ b Type-1 grammar
☐ c Type-2 grammar ☐ d Type-3 grammar

Q.6 Derivation tree is ____.

- ☐ a parse tree ☐ b syntax tree
☐ c both a and b ☐ d none

Q.7 A PDA behaves like FA when the number of auxiliary memory is ____.

- ☐ a 0 ☐ b 1
☐ c 3 ☐ d none

Answer Keys for Fill in the Blanks :

Q.1	Push down automata	Q.2	Accept, reject
Q.3	stack symbol	Q.4	True
Q.5	stack	Q.6	stack

Answer Keys for Multiple Choice Questions :

Q.1	b	Q.2	b	Q.3	d	Q.4	a
Q.5	c	Q.6	a	Q.7	a		



Notes

[illegible]